

Systems Engineering for Deep Learning

Assignment 5

Course Code: CS6886

Aditya Girish

Roll No: EP23B048

B.Tech Engineering Physics

IIT Madras

November 2026

Q1 Your Machine

Machine specifications

The recorded machine specifications are listed below. P and E denote performance and efficiency cores, respectively.

Specification	Value
CPU model	12th Gen Intel Core i7-12650H
Microarchitecture	Alder Lake: Golden Cove (P), Gracemont (E)
Physical / logical cores	10 physical cores / 16 logical cores
Core types	6 Performance cores and 4 Efficiency cores
Base clock	P: 2.30 GHz; E: 1.70 GHz
Maximum single-core clock	P: 4.70 GHz; E: 3.50 GHz
Recorded all-core load clock	P: 1.440 GHz; E: 1.157 GHz
SIMD	AVX2 + FMA3; 256-bit vectors, 8 FP32 elements
FMA units per core	P: 2×256-bit; E: 2×128-bit
Vector registers	16 architectural 256-bit YMM registers per hardware thread
L1 data cache per core	P: 48 KiB; E: 32 KiB
L2 cache	P: 1.25 MiB per core; E: 2 MiB shared by all 4 E-cores
L3 cache	24 MiB shared by all 10 cores
Cache line size	64 bytes
RAM capacity	16 GB
RAM configuration	DDR4-3200 (3200 MT/s), two 64-bit channels
Operating system	Ubuntu 22.04.5 LTS
Compiler	Ubuntu clang 14.0.0-1ubuntu1.1

Derived metrics

An FP32 element occupies 32 bits, and an FMA counts as two FLOPs. Hence, the throughput per cycle is

$$C_P = 2 \times (256/32) \times 2 = 32 \text{ FLOP/cycle},$$
$$C_E = 2 \times (128/32) \times 2 = 16 \text{ FLOP/cycle}.$$

Using the maximum clocks gives the theoretical turbo peaks:

$$P_{\text{one}} = 32 \times 4.70 = \boxed{150.4 \text{ GFLOP/s}} \quad \text{(one P-core)},$$
$$P_{\text{all, upper}} = 6(32 \times 4.70) + 4(16 \times 3.50) = 1126.4 \text{ GFLOP/s}.$$

The all-core value is an optimistic upper bound assuming both core types reach their respective maximum clocks simultaneously. Hardware threads share execution units, so

the calculation counts physical cores. Using the recorded all-core load clocks instead gives

$$P_{\text{all,load}} = 6(32 \times 1.440) + 4(16 \times 1.157) = \boxed{350.528 \text{ GFLOP/s}}.$$

The theoretical memory bandwidth is

$$B = 3200 \times 10^6 \ \frac{\text{transfers}}{\text{s}} \times \frac{64}{8} \ \frac{\text{bytes}}{\text{transfer}} \times 2 = \boxed{51.2 \text{ GB/s}}.$$

The ridge point is $I_{\text{ridge}} = P/B$, giving

$$I_{\text{one}} = 150.4/51.2 = \boxed{2.9375 \text{ FLOP/byte}},$$
$$I_{\text{all}} = 350.528/51.2 = \boxed{6.84625 \text{ FLOP/byte}}.$$

All ridge calculations use the same shared, full-machine memory bandwidth.

Specification	**Value**
Peak FP32 GFLOP/s, one core	150.4 GFLOP/s (one P-core)
Peak FP32 GFLOP/s, all cores	350.528 GFLOP/s
Memory bandwidth (GB/s)	51.2 GB/s
Ridge point, one core (FLOP/B)	2.9375 FLOP/byte
Ridge point, all cores (FLOP/B)	6.84625 FLOP/byte

Q2 Naive Baseline

The recorded naive measurements in `results.csv` are shown below. The implementation is compiled with `clang -O3 -march=native` and runs on one thread. Each reported time is the median of five calls after one warm-up call; input/output file operations are outside the timed region. Following the supplied driver, the two matrix products count as $F = 4N^2d$ FLOPs. Thus,

$$P = \frac{4N^2d}{t_{\text{ms}} \times 10^6} \text{ GFLOP/s}, \qquad \% \text{ peak} = 100\frac{P}{150.4}.$$

The denominator in the formula for peak% is the single-P-core FP32 peak from Q1.

N	d	Time (ms)	GFLOP/s	Peak (%)
1024	64	87.653	3.06	2.04
2048	64	347.811	3.09	2.05
4096	64	1793.042	2.40	1.59
4096	128	4829.885	1.78	1.18

Scaling with N and d. The matrix products require $O(N^2 d)$ work, while softmax requires $O(N^2)$ work. Rearranging the throughput formula gives

$$t_{\text{ms}} = \frac{4N^2 d}{P \times 10^6}, \qquad \frac{t_2}{t_1} = \left(\frac{N_2}{N_1}\right)^2 \frac{d_2}{d_1} \frac{P_1}{P_2}.$$

If throughput P stays constant and matrix-product work dominates, doubling N therefore predicts $4\times$ the time, while doubling d predicts $2\times$ the time. GFLOP/s stays constant under this assumption; it would double only if the work doubled while time stayed fixed. At $d = 64$, doubling N from 1024 to 2048 increases time by $347.811/87.653 = 3.97\times$, close to the expected $4\times$. Doubling N again increases time by $1793.042/347.811 = 5.16\times$, higher than the expected $4\times$.

At $N = 4096$, doubling d from 64 to 128 doubles the matrix-product FLOPs but the time increases by $4829.885/1793.042 = 2.69\times$ instead of doubling; throughput also falls from 2.40 to 1.78 GFLOP/s.

Where the time goes. Both matrix products use a dependent scalar accumulation in their innermost loops, limiting SIMD use and independent FMA execution. In particular, $O = SV$ traverses `V[j*d+k]` with j innermost, accessing a column of a row-major matrix at strides of $4d$ bytes (256 or 512 bytes). This wastes spatial locality, revisits S for each output column, and becomes more costly as d increases. The first product accesses contiguous rows but repeatedly computes separate dot products without blocking. Softmax adds several passes over S, scalar exponentials and divisions. The score matrix S occupies $4N^2$ bytes: 4, 16 and 64 MiB for these three sizes. At $N = 4096$, it exceeds the 24 MiB L3 cache, increasing the cost of storing and revisiting it, which explains the higher cost for $N = 4096$. These access patterns and reductions explain the low 1.18–2.05% peak utilization.

Q3 Optimisations

All variants are single-threaded and use the Q2 timing and FLOP conventions, with peak percentages relative to 150.4 GFLOP/s. The tables use the recorded `results.csv` values, with speedup $= t_{\text{naive}}/t_{\text{variant}}$ using the Q2 baseline for each input.

(a) Loop reordering

Transpose K so score updates access contiguous elements, and reorder the output product to stream rows of V rather than columns. This allows for better spatial locality in the access patterns and allows for cache reuse. Independent output elements become SIMD lanes, removing the naive inner-loop reduction barrier. The remaining limits are repeated accumulator loads/stores, repeated scans of K^T and V, the full $N \times N$ score matrix, and scalar softmax.

N	d	Time (ms)	GFLOP/s	Peak (%)	Speedup (×)
1024	64	17.531	15.31	10.18	5.00
2048	64	81.618	13.16	8.75	4.26
4096	64	323.274	13.29	8.83	5.55
4096	128	551.267	15.58	10.36	8.76

(b) Cache and register tiling

Block both matrix products into 32×64 output tiles with reduction panels of 64 elements. A 4×16 register tile reuses loaded operands across several outputs and retains partial sums in registers within each panel. This improves reuse and reduces accumulator traffic compared with (a). The method is still limited by the full score matrix and separate scalar softmax; partial sums are still loaded/stored between reduction panels. These are the next costs to target.

N	d	Time (ms)	GFLOP/s	Peak (%)	Speedup (×)
1024	64	6.452	41.60	27.66	13.59
2048	64	31.402	34.19	22.73	11.08
4096	64	155.193	27.68	18.40	11.55
4096	128	213.481	40.24	26.75	22.62

(c) Row-wise fusion

For each query, compute its scores, apply softmax, and immediately form its output. Reuse one N-element score buffer: at $N = 4096$, score storage falls from 64 MiB to 16 KiB. This retains (a)'s vector-friendly access but does not use (b)'s register tiling. It still scans K^T and V for every query and retains scalar softmax. Consequently, fusion reduces storage but loses the reuse across query rows provided by tiling; it is seen to be slower than (b) on every input.

N	d	Time (ms)	GFLOP/s	Peak (%)	Speedup (×)
1024	64	22.414	11.98	7.96	3.91
2048	64	90.447	11.87	7.89	3.85
4096	64	303.445	14.15	9.41	5.91
4096	128	533.647	16.10	10.70	9.05

(d) FlashAttention-style tiling with online softmax

Part (a) improved SIMD access, (b) improved reuse across query rows, and (c) removed the full score matrix but lost that reuse. I now combine these benefits using FlashAttention-style blocks (taught in class) in both the query and key dimensions, with stable online softmax.

For each query block, stream key/value tiles while retaining a running row maximum m, normalizer ℓ, and unnormalized output U. For tile scores $S = Q_b K_b^T/\sqrt{d}$, update

$$m' = \max(m, \max S), \quad \alpha = e^{m-m'}, \quad P = e^{S-m'},$$

$$\ell' = \alpha\ell + \sum P, \qquad U' = \alpha U + PV_b.$$

These are row-wise updates; initializing $\ell = 0$, $U = 0$ and use $\alpha = 0$ for the first tile. The final output is normalized once at the end, $O = U/\ell$.

The implementation combines FlashAttention with cache tiling, data packing, register tiling, and SIMD execution via vectorization. It processes 24 query rows against 128 key rows at a time and immediately uses the resulting weights to update the output. Only this 24×128 score tile is stored, requiring 12 KiB instead of the 64 MiB score matrix in (a)/(b) at $N = 4096$. Before processing the tiles, it packs K and V into layouts that make the kernel's vector loads contiguous. These packed arrays are reused across query blocks and require $O(Nd)$ additional storage; packing is included in the measured time.

Both matrix products use a 6×16 AVX2/FMA register kernel. It computes six query rows and sixteen output columns together, reuses loaded operands across those rows, and keeps partial sums in registers throughout each tile's reduction. Each AVX2 instruction operates on eight FP32 elements. For softmax, a local vector exponential approximation evaluates eight score exponentials at once. The tile sizes and kernel shape were selected through recorded tuning.

Softmax reductions, output rescaling and repeated small products remain costly. A recorded profile at $N = 4096, d = 64$ showed that runtime spent about 28 ms in online softmax/rescaling and 18–19 ms in each product.

N	d	Time (ms)	GFLOP/s	Peak (%)	Speedup (×)
1024	64	4.405	60.94	40.52	19.90
2048	64	19.240	55.81	37.11	18.08
4096	64	62.489	68.73	45.70	28.69
4096	128	101.492	84.64	56.27	47.59

Q4 OpenMP

Measurements

Let T_p be the OpenMP time with p threads, T_1 be the naive runtime and T_{serial} the fresh serial FlashAttention time measured in the same batch. I define S_p as the speedup over naive, S_{serial} as the speedup over FlashAttention serial program, and E_p as the parallel efficiency.

$$S_p = \frac{T_1}{T_p}, \qquad E_p = 100\frac{S_p}{p}\%, \qquad S_{\text{serial}} = \frac{T_{\text{serial}}}{T_p}.$$

GFLOP/s uses the same $4N^2d$ convention as Q2.

I evaluate the program at $1, 2, 4, 6, 8, 10, 12$ and 16 threads. Thread placement uses distinct P-cores till six threads, then adds two E-cores at eight threads and all four E-cores at ten, then adds two P-core Simultaneous MultiThreading (SMT) siblings at twelve and all six siblings at sixteen. Thus sixteen workers occupy ten physical cores.

$N = 1024,\ d = 64$

The fresh serial reference is 5.506 ms.

Threads	Time (ms)	GFLOP/s	S_p	E_p (%)	S_{serial}
1	7.412	36.22	1.00	100.00	0.74
2	4.467	60.09	1.66	82.96	1.23
4	2.007	133.75	3.69	92.33	2.74
6	1.361	197.23	5.45	90.77	4.05
8	1.343	199.88	5.52	68.99	4.10
10	2.094	128.19	3.54	35.40	2.63
12	**1.323**	**202.90**	**5.60**	**46.69**	**4.16**
16	2.437	110.15	3.04	19.01	2.26

$N = 2048,\ d = 64$

The fresh serial reference is 22.491 ms.

Threads	Time (ms)	GFLOP/s	S_p	E_p (%)	S_{serial}
1	16.792	63.94	1.00	100.00	1.34
2	8.855	121.26	1.90	94.82	2.54
4	7.330	146.49	2.29	57.27	3.07
6	**3.889**	**276.10**	**4.32**	**71.96**	**5.78**
8	5.066	211.95	3.31	41.43	4.44
10	5.111	210.08	3.29	32.85	4.40
12	5.415	198.29	3.10	25.84	4.15
16	5.971	179.83	2.81	17.58	3.77

$N = 4096,\ d = 64$

The fresh serial reference is 69.440 ms.

Threads	Time (ms)	GFLOP/s	S_p	E_p (%)	S_{serial}
1	77.203	55.63	1.00	100.00	0.90
2	41.293	104.01	1.87	93.48	1.68
4	24.541	175.01	3.15	78.65	2.83
6	24.832	172.96	3.11	51.82	2.80
8	21.721	197.73	3.55	44.43	3.20
10	18.014	238.42	4.29	42.86	3.85
12	20.980	204.72	3.68	30.67	3.31
16	**13.070**	**328.61**	**5.91**	**36.92**	**5.31**

$N = 4096,\ d = 128$

The fresh serial reference is 140.285 ms.

Threads	Time (ms)	GFLOP/s	S_p	E_p (%)	S_{serial}
1	112.942	76.06	1.00	100.00	1.24
2	62.191	138.12	1.82	90.80	2.26
4	42.477	202.23	2.66	66.47	3.30
6	33.644	255.32	3.36	55.95	4.17
8	**32.101**	**267.59**	**3.52**	**43.98**	**4.37**
10	41.713	205.93	2.71	27.08	3.36
12	39.713	216.30	2.84	23.70	3.53
16	34.156	251.49	3.31	20.67	4.11

Scaling and remaining limits

The main computation is highly parallel across query blocks: there are $\lceil N/24 \rceil$ independent blocks giving us 43,86 and 171 blocks for the three values of N. Each block still retains its sequential online-softmax updates across key tiles, but different blocks can execute concurrently. More threads therefore reduce the time spent processing query blocks, while scheduling, barriers, serial allocation and the final uneven distribution of blocks limit the gain.

For a fixed input, Amdahl's law gives the equal-worker reference

$$S_p = \frac{1}{f + (1-f)/p}, \qquad f_{\text{eff}} = \frac{1/S_p - 1/p}{1 - 1/p},$$

where f is the serial fraction under ideal conditions. For $N = 4096, d = 64$, six P-cores give $S_6 = 77.203/24.832 = 3.11$, corresponding to $f_{\text{eff}} \approx 0.186$. For the smaller $N = 1024, d = 64$ input, $S_6 = 5.45$ gives $f_{\text{eff}} \approx 0.020$. The variation shows that one fixed serial fraction does not explain every input. These are fixed-workload measurements, so Amdahl's law makes sense here.

Beyond six threads, the equal-worker assumption becomes weaker. E-cores provide less SIMD throughput than P-cores, and SMT siblings share execution units rather than

adding physical FMA units. Dynamic scheduling allows faster workers to take more blocks, but shared caches, memory traffic and scheduling costs can still limit scaling. The conventional efficiency S_p/p therefore falls partly because the added workers do not provide equal compute capacity.

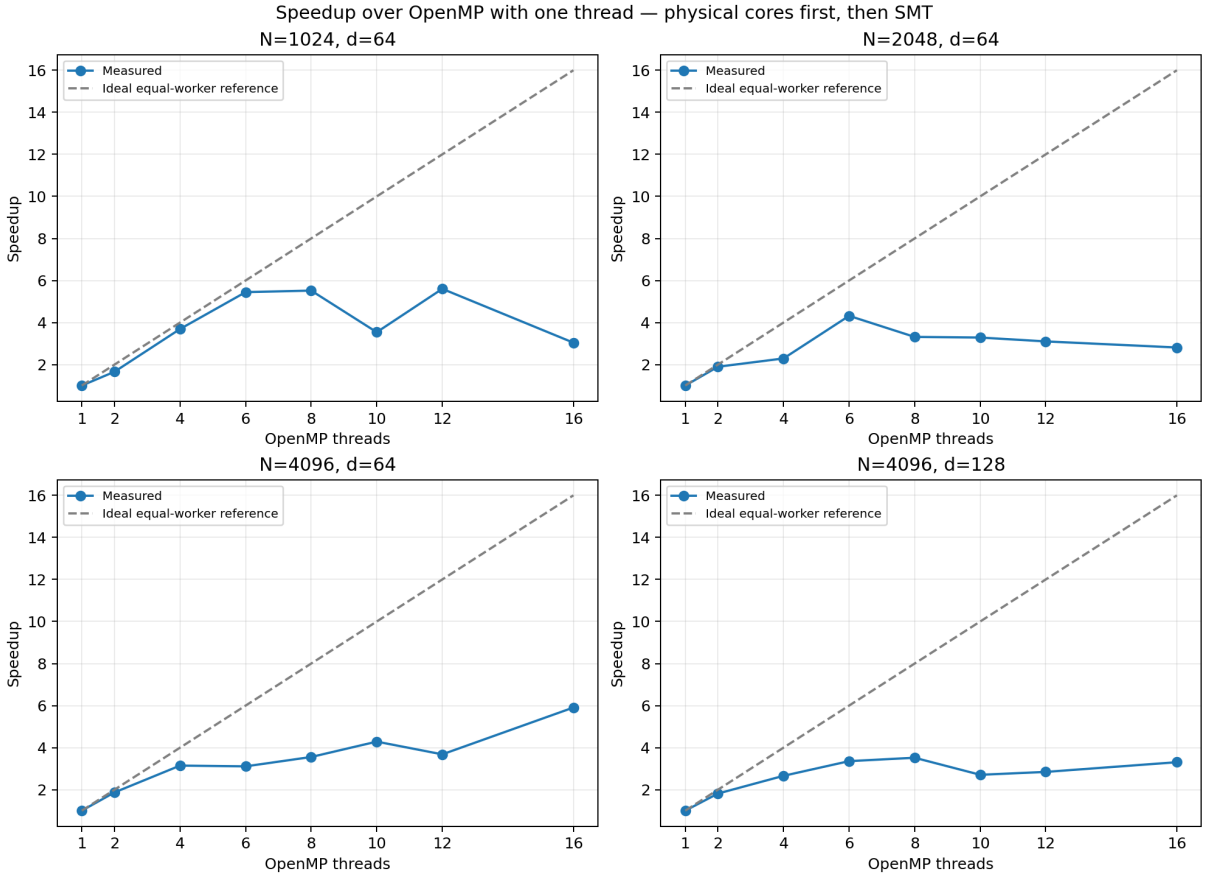


Figure 1: OpenMP speedup relative to one-thread OpenMP for each fixed input. The ideal line assumes equally powerful workers; P/E cores and SMT do not satisfy that assumption.

Q5 Roofline

The roofline uses $N = 4096, d = 64$ and the recorded measurements for every variant and OpenMP thread count. Let F be the throughput, I be the Arithmetic Intensity. Following the driver,

$$F = 4N^2d = 4{,}294{,}967{,}296 \text{ FLOPs}, \qquad I = \frac{F}{D}, \qquad P_{\text{roof}}(I) = \min(P_{\text{peak}}, 51.2I).$$

Here D is the estimated DRAM traffic in bytes and performance is in GFLOP/s. The single-core roof is 150.4 GFLOP/s, with ridge point 2.9375 FLOP/byte. For multicore analysis, I use Q1's measured all-core load clocks of 1.440 GHz for P-cores and 1.157 GHz

for E-cores:

$$P_{\text{all,load}} = 6(32 \times 1.440) + 4(16 \times 1.157) = 350.528 \text{ GFLOP/s}.$$

This gives the main multicore ridge point $350.528/51.2 = 6.84625$ FLOP/byte. The prominent dashed curve shows this load-clock roof. The lighter dotted curve shows the optimistic 1126.4 GFLOP/s all core bound only as an additional reference. The load clocks were measured in Q1's all-core test, rather than during attention, so their roof is a practical estimate rather than an exact ceiling for every run. The 51.2 GB/s theoretical DRAM bandwidth is shared by all workers.

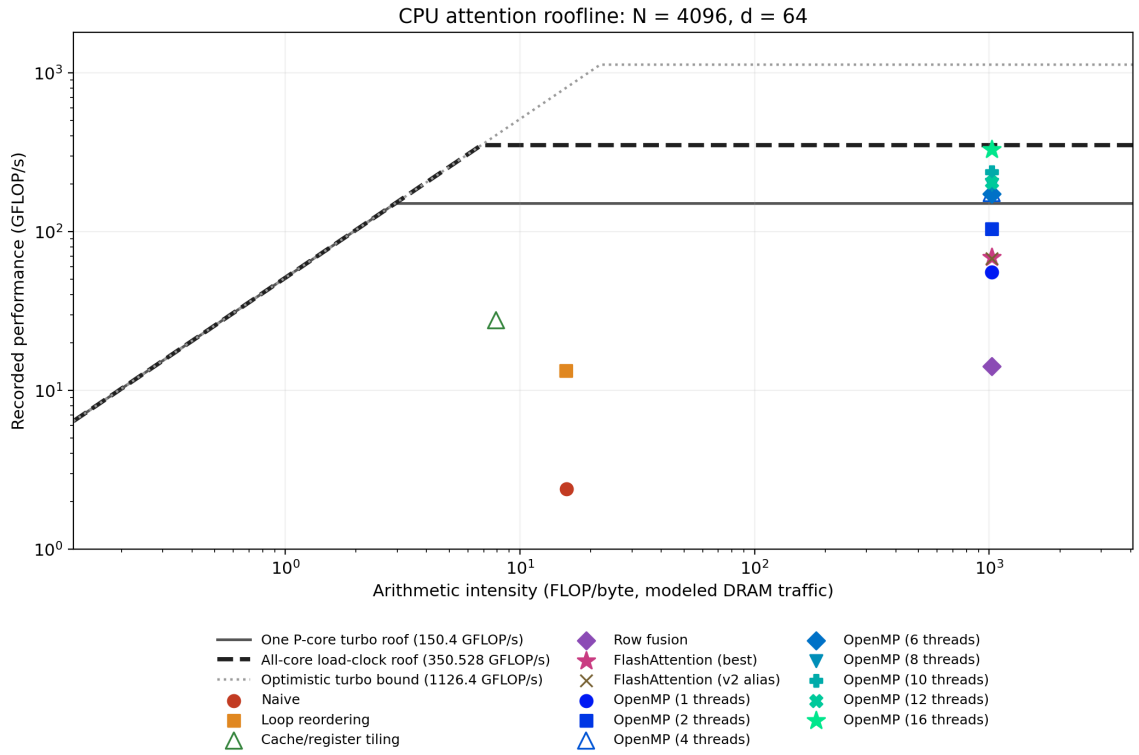


Figure 2: Roofline with FlashAttention and the current OpenMP results. The main multicore roof uses Q1's measured all-core load clocks; Horizontal positions use the analytical DRAM-traffic model below.

Arithmetic intensity and traffic assumptions

I use a cold-data, ideal-cache-reuse model rather than treating every source load as a DRAM access. Each of Q, K, V, O occupies $M = 4Nd = 1{,}048{,}576$ bytes (1 MiB), while the full score matrix occupies $A = 4N^2 = 67{,}108{,}864$ bytes (64 MiB), exceeding the 24 MiB L3 cache. Read each input once and write the output once, giving $4M$ compulsory traffic. Repeated accesses and small temporary buffers are assumed cached; separate

whole-score streaming phases are charged to DRAM. Write allocation, allocator/page-zeroing traffic, conflict misses and temporary eviction after the call are excluded as a simplifying assumption.

Variant	Estimated bytes D	Traffic (MiB)	I (FLOP/byte)
Naive	$4A + 4M$	260	15.754
Loop reordering	$4A + 5M$	261	15.693
Cache/register tiling	$8A + 6M$	518	7.907
Row fusion	$4M$	4	1024
FlashAttention (attn_best)	$4M$	4	1024
OpenMP (all thread counts)	$4M$	4	1024

Naive. Naive writes the scores once, reads and writes them during softmax, then reads them for the output product: 4 times in total, giving $4A$ memory traffic, on top of the compulsory $4M$. Softmax's passes over one 16 KiB row reuse that row in cache.

Loop Reordering. Loop reordering has the same score traffic as the naive; I additionally count one M writeback of the transposed K when the subsequent full-score stream evicts it. This gives, $I_{\text{loops}} = F/(4A + 5M) = 15.693$ FLOP/byte.

Cache/register tiling. The implemented helper first zeroes all scores (A), then reads and writes them for the score product ($2A$). Separate scaling and softmax passes each read and write the scores ($2A$ each), and the output product reads them (A), giving $8A$. At $d = 64$, the score reduction uses one 64-element panel. The initial full-score zeroing also evicts transposed K before its reuse; its writeback and reload contribute $2M$ beyond the $4M$ input/output streams. Thus $I_{\text{tiled}} = F/(8A + 6M) = 7.907$ FLOP/byte.

Row fusion. Row fusion stores one score row, applies softmax to it, and immediately uses it to compute the corresponding output row. This 16 KiB buffer is reused for the next query, so the full 64 MiB score matrix is never written. The row buffer and the 1 MiB transposed K are assumed to remain cached. Only the compulsory input reads and output write are therefore charged to DRAM: $D_{\text{fused}} = 4M$, giving $I_{\text{fused}} = F/(4M) = 1024$ FLOP/byte.

FlashAttention. Like row fusion, FlashAttention consumes scores before they leave cache. It processes 24 query rows against 128 keys at a time, using a 12 KiB score tile and 192 bytes for the running maxima and normalizers. These buffers and the original matrices fit within the 24 MiB L3 cache under the stated reuse assumption, so their repeated accesses do not add modeled DRAM traffic. Thus $D_{\text{Flash}} = 4M$ and $I_{\text{Flash}} = 1024$ FLOP/byte, as for row fusion.

OpenMP. OpenMP applies the same FlashAttention algorithm to independent query blocks. The workers share the packed K/V arrays, while each has its own score tile

and online-softmax state. At sixteen workers, the private buffers occupy 16(12 KiB + 192 bytes) = 195 KiB. Including the shared packed arrays and the four original matrices gives about 6.19 MiB, still below the L3 capacity. Assuming this data remains cached, the compulsory traffic is unchanged: $D_{\text{OMP}} = 4M$ and $I_{\text{OMP}} = 1024$ FLOP/byte for every thread count.

What limits performance

All serial variants lie to the right of the single-core ridge, and all OpenMP points lie to the right of the main multicore ridge at 6.84625 FLOP/byte.

Naive and loop reordering. Naive achieves 2.40 GFLOP/s because dependent reductions and strided V accesses limit SIMD execution and cache-level efficiency. Reordered loops reach 13.29 GFLOP/s at almost the same intensity by exposing contiguous, independent SIMD updates. Repeated accumulator updates and scalar softmax remain limiting costs.

Tiling and row fusion. Tiling reaches 27.68 GFLOP/s through register reuse and more independent accumulators, despite its extra score streams. The full intermediate and scalar softmax remain expensive. Row fusion moves far right by eliminating those streams, but reaches only 14.15 GFLOP/s: processing one query at a time still repeatedly scans operands through cache and lacks the tiled kernel's reuse across query rows.

FlashAttention. FlashAttention combines reuse across queries with a small score tile and vectorized kernels/softmax, reaching 68.73 GFLOP/s, or 45.70% of the single-P-core peak. Online maximum/sum reductions, rescaling output state for every key tile, cache-level operand traffic and kernel overhead leave a gap below the FMA roof.

OpenMP. The updated one-thread point is 55.63 GFLOP/s, and sixteen threads reach 328.61 GFLOP/s (13.070 ms), a 5.91× gain over one-thread OpenMP. Intermediate thread counts are not monotonic, as discussed in Q4. Against the main multicore load-clock roof of 350.528 GFLOP/s, the sixteen-thread result reaches $328.61/350.528 \approx 93.75\%$. It is therefore close to the compute-throughput estimate based on the measured load clocks.